

Database Systems

Relational Algebra



Relational Algebra

✓ Basic Operators

- Derived Operations
- Combining Operators
- Equivalent Expressions
- Operations on Multisets

Relational Algebra

- SQL does not directly use it but incorporates it at its center
- Limitations as useful feature
 - Ease of programming
 - Ability of the compiler to produce highly optimized code
- Algebra
 - Atomic operands (e.g. in arithmetic algebra: variable x or constant **15**)
 - Operators (e.g. in arithmetic algebra: addition, subtraction, etc.)
 - Expression (e.g. in arithmetic algebra: $x + 15 \times (z - 2) = y$)
- Calculus
 - Non-operational, declarative
 - $\{ t \mid C(t) \}$

Basic Operators

- Basic operators
 - **Select:** σ
 - **Project:** π
 - **Union:** $\cup$
 - **Intersection** $\cap$
 - **Set difference:** $-$
 - **Cartesian product:** $\times$
 - **Join:** $\bowtie$
 - **Rename:** ρ
 - **Division:** $/$

Relational Algebra

- Operators
 - **Set operations on relations**
 - Union, Intersection, Difference
 - Operations to **remove** parts of relations
 - **Selection**: Eliminate **rows**
 - **Projection**: Eliminate **columns**
 - Operations that **combine** tuples of two relations
 - Cartesian **product**: pairs the tuples of two relations in all possible ways
 - **Join**: Selectively pair types from two relations
 - **Schema Change**
 - Renaming (change name of relation, attributes)
- Atomic Operands
 - Variables that stand for relations
 - Constants, which are finite relations

Set Operations on Relations

- Two relations **R** and **S**
 - Conditions
 - Identical set of attributes and the types
 - Order columns before operations so order of attributes is the same for both

Union

- $R \cup S$: Set of elements in **R** or **S** or both

Intersection

- $R \cap S$: Set of elements in both **R** and **S**

Difference

- $R - S$: Set of elements in **R** but not in **S**

Set Operations on Relations

Union Example

Relation R:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

Relation S:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

RUS:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

Set Operations on Relations

Intersection Example

Relation R:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

Relation S:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

$R \cap S$:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99

Set Operations on Relations

Difference Example

Relation R:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

Relation S:

name	address	gender	birthdate
Carrie Fisher	12 Maple St., Hollywood	F	9/9/99
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

R-S:

name	address	gender	birthdate
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

Remove Operations

Projection

- Produce from **R** a new relation that has only some of its columns
- $\pi_{A_1, A_2, \dots, A_n}(\mathbf{R})$: a relation that has columns for attributes $A_1, A_2, \dots, A_n$ of **R**
- Resulting schema is set $\{A_1, A_2, \dots, A_n\}$ of attributes

Remove Operations

Projection Example

The relation *Movies*:

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345
Galaxy Quest	1999	104	comedy	DreamWorks	67890
Wayne's World	1992	95	comedy	Paramount	99999

$\pi_{\text{title, year, length}}(\textit{Movies})$:

title	year	length
Star Wars	1977	124
Galaxy Quest	1999	104
Wayne's World	1992	95

Remove Operations

Selection

- Produce from **R** a new relation with subset of **R**'s tuples
- $\sigma_C(\mathbf{R})$: a relation with tuples from **R** that satisfy some condition **C** that involves the attributes of **R**
- **C** is a conditional expression with constant or attribute operands
- Schema in the resulting relation is the same as **R**'s schema

Remove Operations

- Selection Example

The relation *Movies*:

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345
Galaxy Quest	1999	104	comedy	DreamWorks	67890
Wayne's World	1992	95	comedy	Paramount	99999

$\sigma_{length \geq 100}(Movies)$:

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345
Galaxy Quest	1999	104	comedy	DreamWorks	67890

Remove Operations

- Selection Example

The relation *Movies*:

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345
Galaxy Quest	1999	104	comedy	DreamWorks	67890
Wayne's World	1992	95	comedy	Paramount	99999

$\sigma_{length \geq 100 \text{ AND } studioName = 'Fox'}(Movies)$:

title	year	length	genre	studioName	producerC#
Star Wars	1977	124	sciFi	Fox	12345

Combine Operations

Cartesian Product (Cross-Product, or Product)

- Is a set of pairs that can be formed by choosing the first element of the pair to be **any** element of relation **R** and the second **any** element of relation **S**
- **RXS**: Members of **R** and **S** are tuples, the result is a longer tuple with one component for each of the components of the constituent tuples
- Resulting relation schema is the union of **R** and **S** schemas
 - If attribute **A** in common, **R.A** and **S.A** used to disambiguate

Combine Operations

- Cartesian Product Example

The relation R :

A	B
1	2
3	4

The relation S :

B	C	D
2	5	6
4	7	8
9	10	11

The relation $R \times S$:

A	R.B	S.B	C	D
1	2	2	5	6
1	2	4	7	8
1	2	9	10	11
3	4	2	5	6
3	4	4	7	8
3	4	9	10	11

Combine Operations

- Join
 - Choosing only **one** set of tuples to pair from the product $\mathbf{R} \times \mathbf{S}$
 - Natural Join $\mathbf{R} \bowtie \mathbf{S}$
 - Pairing on common attributes
 - Theta Join $\mathbf{R} \bowtie_C \mathbf{S}$
 - Pair tuples using one specific condition $\mathbf{C}$

Combine Operations: Natural Join

- $\mathbf{R} \bowtie \mathbf{S}$
 - Pair only those tuples of $\mathbf{R}$ and $\mathbf{S}$ that agree in whatever attributes are common to the schemas of $\mathbf{R}$ and $\mathbf{S}$
 - A tuple $\mathbf{r}$ and a tuple $\mathbf{s}$ are matched if and only if $\mathbf{r}$ and $\mathbf{s}$ agree on value on each of attributes $\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_n$ that are in both schemas
 - The resulting tuple is called *joined tuple*

Combine Operations

- Natural Join Example

The relation R :

A	B	C
1	2	3
6	7	8
9	7	8

The relation S :

B	C	D
2	3	4
2	3	5
7	8	10

The relation $R \bowtie S$:

A	B	C	D
1	2	3	4
1	2	3	5
6	7	8	10
9	7	8	10

Combine Operations: Natural Join

- Dangling Tuple

- A tuple that fails to pair with **any tuple of the other relation** in a join is said to be a *dangling tuple*.
- Does not have pair with any tuple and this has no effect on the result of natural join

- Example: Dangling tuple (9, 10, 11)

The relation R :

A	B
1	2
3	4

The relation S :

B	C	D
2	5	6
4	7	8
9	10	11

The relation $R \bowtie S$:

A	B	C	D
1	2	5	6
3	4	7	8

Combine Operations: Theta Join

- $R \bowtie_C S$
 - Pair tuples using one specific condition C
 - Traditionally Theta referred to the condition (we now use C)
 - Procedure
 - Take the product of R and S
 - Select from the product only those tuples that satisfy the condition C
- Schema is union of schemas of R and S

Combine Operations

- Theta Join Example

The relation R :

A	B	C
1	2	3
6	7	8
9	7	8

The relation S :

B	C	D
2	3	4
2	3	5
7	8	10

The relation $R \bowtie_{A < D} S$:

A	R.B	R.C	S.B	S.C	D
1	2	3	2	3	4
1	2	3	2	3	5
1	2	3	7	8	10
6	7	8	7	8	10
9	7	8	7	8	10

Combine Operations

- Theta Join Example

The relation R :

A	B	C
1	2	3
6	7	8
9	7	8

The relation S :

B	C	D
2	3	4
2	3	5
7	8	10

The relation $R \bowtie_{A < D \text{ AND } R.B \neq S.B} S$:

A	R.B	R.C	S.B	S.C	D
1	2	3	7	8	10

Schema Change Operations

- Renaming
 - $\rho_S(A_1, A_2, \dots, A_n)(R)$ renames the attributes **A1, A2, ... , An** of the relation **R**
 - The resulting relation has exactly the same tuples of the relation **R**
 - The name of the relation is **S**
 - The attributes are named **A1, A2, ... , An** from the left
 - $\rho_S(R)$ only changes the name of the relation to **S** and leaves the attributes as they are in **R**

Schema Change Operations

- Renaming

The relation R :

A	B
1	2
3	4

The relation S :

B	C	D
2	5	6
4	7	8
9	10	11

The relation $R \bowtie \rho_{S(X, C, D)}(S)$:

A	B	X	C	D
1	2	2	5	6
1	2	4	7	8
1	2	9	10	11
3	4	2	5	6
3	4	4	7	8
3	4	9	10	11

Relationship Among Operations

- $R \cap S = R - (R - S)$
- $R \bowtie_C S = \sigma_C (R \times S)$
- $R \bowtie S = \pi_L(\sigma_C (R \times S))$

Relationship Among Operators

- Select: σ
- Project: π
- Union: $\cup$
- Intersection $\cap$
- Set difference: $-$
- Cartesian product: $\times$
- Join: $\bowtie$
- Rename: ρ
- Division: $/$

Schemas For Results

- **Union, intersection, and difference:** The schemas of the two operands must be the same, so use that schema for the result
- **Selection:** Schema of the result is the same as the schema of the operand
- **Projection:** List of attributes tells us the schema.
- **Renaming:** The operator tells the schema

Schemas for Results

- **Product:** Schema is the attributes of both relations. Use **R.A**, etc., to distinguish two attributes named A
- **Theta-join:** Same as product
- **Natural join:** Union of the attributes of the two relations.

Relational Algebra

- Basic

✓ Derived Operators

- Combining Operators
- Equivalent Expressions
- Operations on Multisets

Derived Operators

- Intersection $\cap$
- Join: $\bowtie$
 - Natural Join
 - Theta Join
 - Equijoin
 - Inner Join
 - Outer Join
- Divide: $/$

Intersection

$$\mathbf{R} \cap \mathbf{S} = \mathbf{R} - (\mathbf{R} - \mathbf{S})$$

- First subtracting $\mathbf{S}$ from $\mathbf{R}$ to form a relation $\mathbf{T}$ consisting of all those tuples in $\mathbf{R}$ but not $\mathbf{S}$
- Subtract $\mathbf{T}$ from $\mathbf{R}$, leaving only those tuples of $\mathbf{R}$ that are also in $\mathbf{S}$

Division

R/S

- Find tuples in R that match **all** tuples in another relation

$$\mathbf{R/S} = \pi_{(\mathbf{R-S})} (\mathbf{R}) - \pi_{(\mathbf{R-S})} [(\pi_{(\mathbf{R-S})} (\mathbf{R}) \times \mathbf{S}) - \mathbf{R}]$$

- **R** relation with attributes $\mathbf{A_1, \dots, A_n, B_1, \dots, B_m}$
- **S** relation with attributes $\mathbf{B_1, \dots, B_m}$ (a subset of **R**'s attributes)
- **R/S** with attributes $\mathbf{A_1, \dots, A_n}$ is a relation with attributes that consist of all tuples that
 - For every tuple with attributes $\mathbf{A_1, \dots, A_n}$ concatenated with a tuple with attributes $\mathbf{B_1, \dots, B_m}$ with values in **S** the concatenated tuple with attributes $\mathbf{A_1, \dots, A_n, B_1, \dots, B_m}$ is in **R**

Division Example

The relation **R**:

Student	Task
Fred	Database1
Fred	Database2
Fred	Compiler1
Eugene	Database1
Eugene	Compiler1
Sarah	Database1
Sarah	Database2

The relation **S**:

Task
Database1
Database2

The relation **R/S**:

Student
Fred
Sarah

Division Derivation In Detail

$$\mathbf{R/S} = \pi_{(\mathbf{R-S})}(\mathbf{R}) - \pi_{(\mathbf{R-S})}[(\pi_{(\mathbf{R-S})}(\mathbf{R}) \times \mathbf{S}) - \mathbf{R}]$$

- Start with computing a table with attributes $\mathbf{A}_1, \dots, \mathbf{A}_n$ from $\mathbf{R}$: it will be $\pi_{(\mathbf{R-S})}(\mathbf{R})$
- Then find the product of that table with table $\mathbf{S}$:
it will be $(\pi_{(\mathbf{R-S})}(\mathbf{R}) \times \mathbf{S})$
- This $(\pi_{(\mathbf{R-S})}(\mathbf{R}) \times \mathbf{S})$ table will have attributes $\mathbf{A}_1, \dots, \mathbf{A}_n, \mathbf{B}_1, \dots, \mathbf{B}_m$ as a result of the product.
It's attributes $\mathbf{B}_1, \dots, \mathbf{B}_m$ will have values from $\mathbf{S}$

Division Derivation In Detail

- Then we remove from that table, any tuple that is in initial table **R**: it will be $[(\pi_{(R-S)}(\mathbf{R}) \times S) - \mathbf{R}]$
- Finally, we remove all tuples $\mathbf{A}_1, \dots, \mathbf{A}_n$ that are in that product, but not in the $\mathbf{A}_1, \dots, \mathbf{A}_n$ of **R**, from the **R**'s projection of $\mathbf{A}_1, \dots, \mathbf{A}_n$ with
$$\mathbf{R}/S = \pi_{(R-S)}(\mathbf{R}) - \pi_{(R-S)}[(\pi_{(R-S)}(\mathbf{R}) \times S) - \mathbf{R}]$$
- The last two steps are similar to derivation of intersect

Derived Division Example

The relation R :

Student	Task
Fred	Database1
Fred	Database2
Fred	Compiler1
Eugene	Database1
Eugene	Compiler1
Sarah	Database1
Sarah	Database2

The relation S :

Task
Database1
Database2

1

The relation $\pi_{(R-S)}(R)$

Student
Fred
Eugene
Sarah

2

The relation $(\pi_{(R-S)}(R) \times S)$

Student	Task
Fred	Database1
Fred	Database2
Eugene	Database1
Eugene	Database2
Sarah	Database1
Sarah	Database2

Derived Division Example

3

$$[(\pi_{(R-S)}(R) \times S) - R]$$

Student	Task
Eugene	Database2

6

$$R/S = \pi_{(R-S)}(R) - \pi_{(R-S)}[(\pi_{(R-S)}(R) \times S) - R]$$

Student
Fred
Sarah

4

$$\pi_{(R-S)}[(\pi_{(R-S)}(R) \times S) - R]$$

Student
Eugene

5

$$\pi_{(R-S)}(R) - \pi_{(R-S)}[(\pi_{(R-S)}(R) \times S) - R]$$

Student	-	Student
Fred		Eugene
Eugene		
Sarah		

Theta Join

- $R \bowtie_C S = \sigma_C(R \times S)$
 - Start with the product $R \times S$
 - Then apply the selection operator with a condition C

Natural Join

- $\mathbf{R} \bowtie \mathbf{S} = \pi_L(\sigma_C(\mathbf{R} \times \mathbf{S}))$

- Start with the product $\mathbf{R} \times \mathbf{S}$
- Then apply the selection operator with a condition C of the form

$R. A_1 = S. A_1$ **AND** $R. A_2 = S. A_2$ **AND** $R. A_3 = S. A_3$... **AND** $R. A_n = S. A_n$
where $A_1, \dots, A_n$ are all the attributes appearing in the schemas of both $\mathbf{R}$ and $\mathbf{S}$

- Then we project out one copy of each of the equated attributes.
 $\mathbf{L}$: list of attributes in $\mathbf{R}$ followed by attributes in $\mathbf{S}$ that are not in $\mathbf{R}$

Equijoin

- Theta join
- Join condition C consists only of equalities

$$\mathbf{R} \bowtie_C \mathbf{S} = \sigma_C (\mathbf{R} \times \mathbf{S})$$

- Start with the product $\mathbf{R} \times \mathbf{S}$
- Then apply the selection operator with a condition C
- C consists only of equalities

Joins

- Inner Join
 - Includes only those tuples with matching attributes and the rest are discarded in the resulting relation
 - Natural Join
 - Theta Join
 - Equijoin
- Outer Join
 - Include unmatched tuples from the participating relations
 - Full outer join
 - Left outer join
 - Right outer join

Outer Join

- The outer join (full outer join) $\mathbf{R} \bowtie_C^{\text{outer}} \mathbf{S}$
 - Schema contains union of the (possibly renamed) attributes in $\mathbf{R}$ and $\mathbf{S}$
 - Tuples consist of
 - The tuples in the regular join
 - The tuples of $\mathbf{R}$ that do not join with any tuple in $\mathbf{S}$, and padded with **NULL** instead
 - The tuples of $\mathbf{S}$ that do not join with any tuple in $\mathbf{R}$, and padded with **NULL** instead

Left Outer Join

- Left outer join $\mathbf{R} \overset{\text{left}}{\bowtie}_C \mathbf{S}$
 - Schema contains union of the (possibly renamed) attributes in $\mathbf{R}$ and $\mathbf{S}$
 - Tuples consist of
 - The tuples in the regular join
 - The tuples of $\mathbf{R}$ that do not join with any tuple in $\mathbf{S}$, and padded with **NULL** instead

Right Outer Join

- Right outer join $\mathbf{R} \bowtie_C^{\text{right}} \mathbf{S}$
 - Schema contains union of the (possibly renamed) attributes in $\mathbf{R}$ and $\mathbf{S}$
 - Tuples consist of
 - The tuples in the regular join
 - The tuples of $\mathbf{S}$ that do not join with any tuple in $\mathbf{R}$, and padded with **NULL** instead

Outer Join Example

The relation R :

A	B
1	2
3	4

The relation S :

B	C	D
2	3	4
5	6	7

The relation $R \bowtie_{R.B=S.B}^{outer} S$:

A	R.B	S.B	C	D
1	2	2	3	4
3	4	NULL	NULL	NULL
NULL	NULL	5	6	7

Left Outer Join Example

The relation R :

A	B
1	2
3	4

The relation S :

B	C	D
2	3	4
5	6	7

The relation $R \bowtie_{R.B=S.B}^{left} S$:

A	R.B	S.B	C	D
1	2	2	3	4
3	4	NULL	NULL	NULL

Right Outer Join Example

The relation **R**:

A	B
1	2
3	4

The relation **S**:

B	C	D
2	3	4
5	6	7

The relation $\mathbf{R} \bowtie_{R.B=S.B}^{\text{right}} \mathbf{S}$:

A	R.B	S.B	C	D
1	2	2	3	4
NULL	NULL	5	6	7

Relational Algebra as a Constraint Language

- Two ways to express
 - $\mathbf{R}$ is an expression of relational algebra
 $\mathbf{R} = \mathbf{0}$ is a constraint
Meaning: The value of $\mathbf{R}$ must be empty (no tuples in the result of $\mathbf{R}$)
 - $\mathbf{R}$ and $\mathbf{S}$ are expressions of relational algebra
 $\mathbf{R} \subseteq \mathbf{S}$ is a constraint
Meaning: Every tuple in the result of $\mathbf{R}$ must also be in the result of $\mathbf{S}$
The result of $\mathbf{S}$ may contain additional tuples not produced by $\mathbf{R}$.

Referential Integrity Constraints

- Asserting a value appearing in one context also appears in another, related context
- Example
 - `Movies(title, year, length, genre, studioName, producerC#)`
 - `StarsIn(movieTitle, movieYear, starName)`
- If we want to assert that any movie mentioned in the relation StarsIn also appears in the relation Movies

$$\pi_{movieTitle, movieYear}(StarsIn) \subseteq \pi_{title, year}(Movies)$$

Key Constraint

- Example

- The relation movie star with name attribute as key

MovieStar(name, address, gender, birthdate)

- If we construct all pairs of **MovieStar** tuples (**t1**, **t2**), we must not find a pair that agree in the name component and disagree in address components
 - To construct the pairs we use a Cartesian product
 - To search for pairs that violate the condition we use a selection
 - We then assert the constraint by equating the result to **0**

Key Constraint

$$\sigma_{MS1.name=MS2.name \textbf{ AND } MS1.address \neq MS2.address} (MS1 \times MS2) = 0$$

MS1 in the product $MS1 \times MS2$ is shorthand for the renaming:

$$\rho_{MS1(name, address, gender, birthdate)}(MovieStar)$$

Additional Operators

- There are additional relational algebra operators
 - Usually used in the context of **query optimization**
- Duplicate elimination – δ
 - Used to turn a bag into a set
- Aggregation operators
 - e.g. sum, average
- Grouping – γ
 - Used to partition tuples into groups
 - Typically used with aggregation

Relational Algebra

- Basic Operators
- Derived Operators
- ✓ **Combining Operators**
- Equivalent Expressions
- Operations on Multisets

Combining Operators to Form Queries

- Expressions by applying operations to the result of other operations.
 - Parentheses and precedence rules
- Three notations
 - Expression trees
 - Expressions with several operators
 - Sequences of assignment statements

Expressions

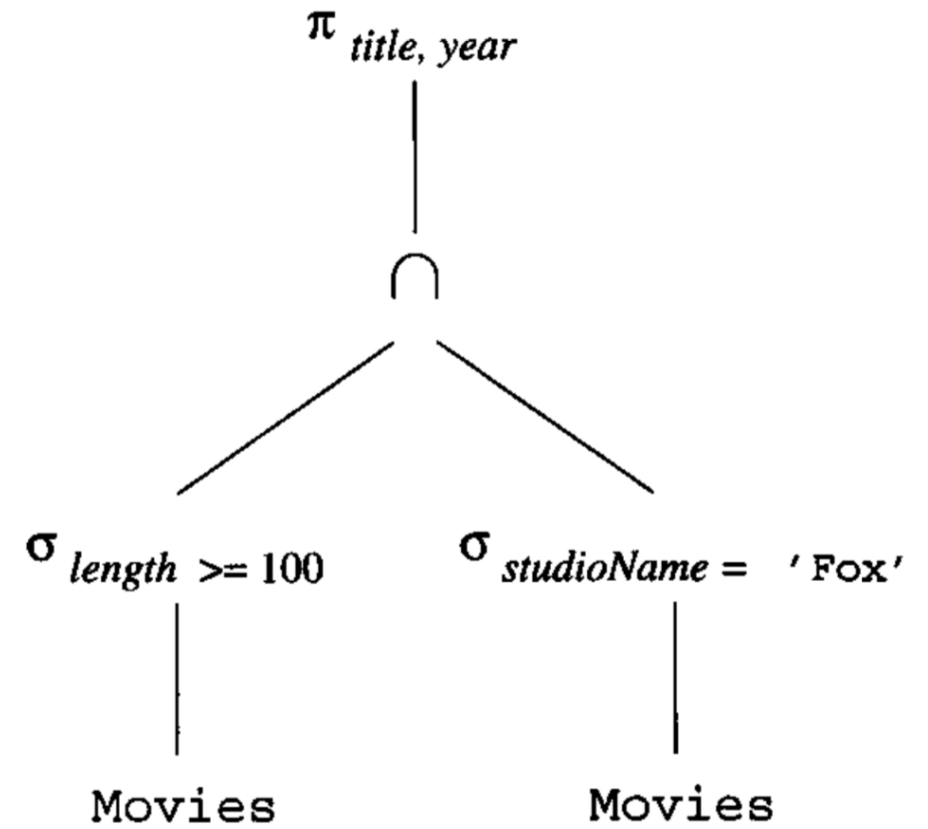
- **Example**

What are the titles and years of movies made by **Fox** that are at least **100** minutes long?

1. Select those Movies tuples that have $length \geq 100$
2. Select those Movies tuples that have $studioName = 'Fox'$
3. Compute the intersection of (1) and (2)
4. Project the relation from (3) onto attributes title and year

Tree Expression

- Evaluated bottom-up
 - Applying the operator at an interior node to the arguments, which are the results of its children
- Example
 - The two selection nodes: steps (1) and (2)
 - The intersection node: step (3)
 - The projection node is step (4)



Linear Expression

- More than one relational algebra expression that represents the same computation

$$\pi_{title,year}(\sigma_{length \geq 100}(Movies) \cap \sigma_{studioName = 'Fox'}(Movies))$$

$$\pi_{title,year}(\sigma_{length \geq 100 \text{ AND } studioName = 'Fox'}(Movies))$$

Linear Notation

- Names for temporary relations corresponding to interior nodes of the tree
- Sequence of assignments that create value for each temporary relation
- Flexible order as long as value ready

Linear Notation

- A relation name and parenthesized list of attributes for that relation. The name answer will be used conventionally for the result of the final step
 - i.e. the name of the relation at the root of the expression tree
- The assignment symbol ($:=$)
- Any algebraic expression on the right

Linear Notation

- $R(t, y, l, i, s, p) := \sigma_{length \geq 100}(Movies)$
- $S(t, y, l, i, s, p) := \sigma_{studioName = 'Fox'}(Movies)$
- $Answer(title, year) := \pi_{t,y}(R \cap S)$

Relational Algebra (2)

- Review Discussed Operators
- More on Derived Operators
- Combining Operators
- ✓ **Equivalent Expressions**
- Operations on Multisets

Equivalent Expressions & Query Optimization

- Query based on expressions similar to relational algebra
- Query asked may have many equivalent expressions
 - Some much more quickly evaluated
- Query optimization: replace one expression of relational algebra by an equivalent expression that is more efficiently evaluated

Equivalent Expressions & Query Optimization

- Semantic equivalence: results are *always* the same

$$\pi_{title,year}(\sigma_{length \geq 100}(Movies) \cap \sigma_{studioName = 'Fox'}(Movies))$$

$$\pi_{title,year}(\sigma_{length \geq 100 \text{ AND } studioName = 'Fox'}(Movies))$$

- Questions to ask
 - Are they equivalent?
 - Which one is more efficient?
 - Can you make it even more efficient?

Relational Algebra

- Basic Operators
- Derived Operators
- Combining Operators
- Equivalent Expressions
- ✓ Operations on Multisets

Operations on Multisets

- A bag (or multiset) is like a set, but an element may appear more than once
- Example: $\{1,2,1,3\}$ is a bag
- Example: $\{1,2,3\}$ is also a bag that happens to be a set

Operations on Multisets

- SQL, the most important query language for relational databases, is actually a **bag** language
- Some operations, like projection, are more efficient on bags than sets

Operations on Multisets

- Selection applies to each tuple, so its effect on bags is like its effect on sets
- Projection also applies to each tuple, but as a bag operator, we do not eliminate duplicates
- Products and joins are done on each pair of tuples, so duplicates in bags have no effect on how we operate

Bag Union

- An element appears in the union of two bags the sum of the number of times it appears in each bag
- Example: $\{1,2,1\} \cup \{1,1,2,3,1\} = \{1,1,1,1,1,2,2,3\}$

Bag Intersection

- An element appears in the intersection of two bags the minimum of the number of times it appears in either
- Example: $\{1,2,1,1\} \cap \{1,2,1,3\} = \{1,1,2\}$

Bag Difference

- An element appears in the difference $\mathbf{A} - \mathbf{B}$ of bags as many times as it appears in $\mathbf{A}$, minus the number of times it appears in $\mathbf{B}$
- But never less than $\mathbf{0}$ times
- Example: $\{1,2,1,1\} - \{1,2,3\} = \{1,1\}$

Bag Laws are not Set Laws

- Some, but not all algebraic laws that hold for sets also hold for bags
- Example: the commutative law for union ($\mathbf{R} \cup \mathbf{S} = \mathbf{S} \cup \mathbf{R}$) does hold for bags
- Since addition is commutative, adding the number of times x appears in $\mathbf{R}$ and $\mathbf{S}$ does not depend on the order of $\mathbf{R}$ and $\mathbf{S}$

Law that Fails

- Set union is idempotent, meaning that $S \cup S = S$
- However, for bags, if x appears n times in S , then it appears $2n$ times in $S \cup S$
- Thus $S \cup S \neq S$ in general
- e.g., $\{1\} \cup \{1\} = \{1,1\} \neq \{1\}$

Summary

- Relational Algebra operators
 - Five core operators: selection, projection, cross-product, union and set difference
 - Additional operators are defined in terms of the core operators
 - E.g. Intersection, join
- SQL and Relational Algebra can express the same class of queries
- Multiple Relational Algebra queries can be equivalent
 - Same semantics but different performance
 - Form basis for optimizations

Acknowledgements

[1] Database Systems: The Complete Book, 2nd Edition
Hector Garcia-Molina, Jeffrey D. Ullman, Jennifer Widom
Prentice Hall, 2009

[2] Database System Concepts, Seventh Edition
Avi Silberschatz, Henry F. Korth, S. Sudarshan
McGraw-Hill, March 2019
www.db-book.com

[3] CMPT 354 (Jiannan Wang - SFU)
<https://sfu-db.github.io/cmpt354/>

Additional references and resources used in preparation of this course are listed on
<https://canvas.sfu.ca/courses/13083/pages/references>
or mentioned on each slide.